



School: Campus:
Academic Year: Subject Name: Subject Code:
Semester: Program: Branch: Specialization:
Date:

Applied and Action Learning

(Learning by Doing and Discovery)

Name of the Experiment :

* Coding Phase: Pseudo Code / Flow Chart / Algorithm

Coding Phase: Pseudo Code / Flow Chart / Algorithm

1. A Peer-to-Peer (P2P) network is a decentralized architecture where every node has equal authority.
2. Each node functions both as a client and a server, enabling direct data sharing without intermediaries.
3. Unlike centralized systems, there is no single controlling entity in a P2P network.
4. This decentralized nature enhances transparency, reliability, and fault tolerance.
5. In blockchain technology, the P2P network forms the backbone of distributed ledger systems.
6. Every blockchain node maintains a complete or partial copy of the blockchain data.
7. When a user initiates a transaction, it is broadcast to nearby peer nodes in the network.
8. These peers then verify and propagate the transaction further across the network.
9. The process ensures that all participants receive and validate the same data simultaneously.
10. Blocks containing verified transactions are also distributed through the P2P network.
11. Each node checks the validity of new blocks before adding them to its local ledger copy.
12. Consensus mechanisms such as Proof of Work and Proof of Stake function within this network.
13. These mechanisms help all nodes agree on a single, consistent version of the blockchain.
14. The P2P structure eliminates the risk of a single point of failure.

15. It makes the network more resistant to censorship, attacks, and data manipulation.
16. Peer discovery enables nodes to locate and connect with other nodes dynamically.
17. Data synchronization ensures that every node maintains an up-to-date blockchain record.
18. The distributed communication model improves both scalability and reliability.
19. information exchange in P2P networks occurs through message broadcasting and block propagation.
20. Nodes continuously communicate to validate, store, and share new blockchain information.

*** Softwares used**

- Network simulation tools or libraries (e.g., peersim, libp2p)
- MetaMask
- Local test environment or virtual machines for distributed node

* Implementation Phase: Final Output (no error)

Applied and Action Learning

The implementation of a Peer-to-Peer (P2P) network in blockchain involves creating multiple interconnected nodes that communicate directly without any central server. Each node can send, receive, and forward messages independently. Peer discovery protocols are established to help nodes identify and connect with others dynamically. A transaction or block generated by one node is broadcast and relayed across the network until all nodes receive it. Network conditions such as latency or node failure are simulated to test system robustness. Nodes verify received data to ensure accuracy and synchronization. Consensus logic is optionally added to mimic blockchain validation. Network monitoring confirms message propagation and consistency. The overall setup demonstrates decentralization, reliability, and transparency. This reflects how real blockchain networks maintain secure and synchronized ledgers.

* Observations

- Multiple nodes were successfully created and connected in a decentralized P2P network.
- Peer discovery protocols enabled nodes to dynamically find and connect with others.
- The network remained stable and resilient under latency and node failure conditions.
- The simulation proved that the P2P structure supports decentralization, transparency, and fault tolerance in blockchain systems.

ASSESSMENT

Rubrics	Full Mark	Marks Obtained	Remarks
Concept	10		
Planning and Execution/ Practical Simulation/ Programming	10		
Result and Interpretation	10		
Record of Applied and Action Learning	10		
Viva	10		
Total	50		

Signature of the Student:

Name :

Signature of the Faculty:

Regn. No. :

Page No.....

**As applicable according to the experiment.
Two sheets per experiment (10-20) to be used.*